MEMORIA PROYECTO: Buscando al Coronel Kurtz

Míriam Mingo Revuelta

2º B iMAT

PARTE 1: Agente inteligente en un entorno parcialmente observable (kurtz.py)

El objetivo de esta sección es diseñar e implementar un agente inteligente, el Capitán Willard, capaz de explorar un palacio representado como una cuadrícula 6x6 con el fin de encontrar al coronel Kurtz y, posteriormente, abandonar el palacio con vida.

El entorno es **parcialmente observable**, ya que el agente no conoce inicialmente la disposición de los peligros (precipicios y soldado enemigo), ni la posición de Kurtz ni de la salida. Por lo tanto, Willard debe percibir, razonar y actuar basándose exclusivamente en la información que va obteniendo a lo largo de la exploración.

El agente sigue el ciclo **percepción-razonamiento-acción**. En cada iteración, primero percibe el entorno inmediato, después actualiza su base de conocimiento mediante inferencias lógicas y, finalmente, decide la siguiente acción a realizar en función del conocimiento acumulado.

Modelado del entorno y perceptos

Inicialmente, se genera la matriz que representa el palacio donde cada celda puede contener: a Willard (en (0,0)), un precipicio, un soldado enemigo, al Coronel Kurtz, la salida o estar vacía. Tras colocar a Willard, el resto de los elementos se sitúan de forma aleatoria al inicio de la ejecución, utilizando una lista de posiciones libres que se mezcla y de la cual se van extrayendo posiciones para evitar solapamientos.

En cada paso, el agente recibe un conjunto de perceptos asociados a las celdas adyacentes a su posición actual (arriba, abajo, izquierda y derecha): **brisa** (presencia de precipicio), **ronquido** (soldado enemigo), **resplandor** (proximidad de la salida), información sobre la existencia de **paredes** (límites del tablero) y un grito (en caso de haber eliminado al soldado enemigo con una granada).

```
return [brisa, ronquido, resplandor, pared_arriba, pared_abajo, pared_izqda, pared_dcha, int(grito)]
```

Inferencias y razonamiento lógico

Para razonar sobre el entorno, Willard mantiene internamente varias estructuras que le permiten acumular conocimiento y utilizarlo para realizar inferencias:

- Una **matriz de celdas seguras**, donde cada celda puede ser segura (True), peligrosa con certeza (False) o desconocida (None).
- Una **matriz de celdas visitadas**.
- Un **historial de perceptos**, que almacena todas las posiciones visitadas junto con los perceptos percibidos en ellas.
- Conjuntos de posibles posiciones de precipicios y del soldado enemigo.

El razonamiento del agente se basa en reglas lógicas derivadas de los perceptos. Por un lado, se aplican **inferencias directas**. Por ejemplo, si en una celda no se percibe brisa ni

ronquido, todas sus celdas adyacentes se marcan como seguras. En caso contrario, estas se consideran posibles ubicaciones del peligro correspondiente.

Por otro lado, Willard realiza **inferencias acumulativas** haciendo uso del historial completo de perceptos que ha ido guardando. Para cada celda del mapa, se comprueba si pudiera contener un peligro analizando posibles contradicciones con perceptos previos. Dado que los preceptos solo afectan a celdas adyacentes, determinaremos si lo son mediante la **distancia Manhattan** ($|x - h_x| + |y - h_y| = 1$). Si una celda es adyacente a una posición donde no se percibió brisa o ronquido, se descarta que contenga un precipicio o un soldado respectivamente. De este modo, una celda solo se marca como segura cuando todos los peligros posibles han sido descartados lógicamente, evitando asumir seguridad por ausencia de información.

El mapa mostrado al usuario no representa el estado real del entorno, sino el **conocimiento interno del agente**, diferenciando entre celdas seguras, peligrosas y desconocidas según las inferencias realizadas hasta ese momento.

Ejecución

Se dispone de dos modos de ejecución:

- **Modo manual.** En el que el usuario controla directamente los movimientos del agente, pero dispone de la información inferida (mapa de seguros) y de los perceptos para tomar decisiones informadas.

```
Mapa conocido:
[·][·][?][?][?][?]
[·][·][·][?][?][?]
[·][·][·][·][?][?]
[·][·][CW][!][?][?]
[?][·][!][?][?][?]
[?][?][?][?][?][?]

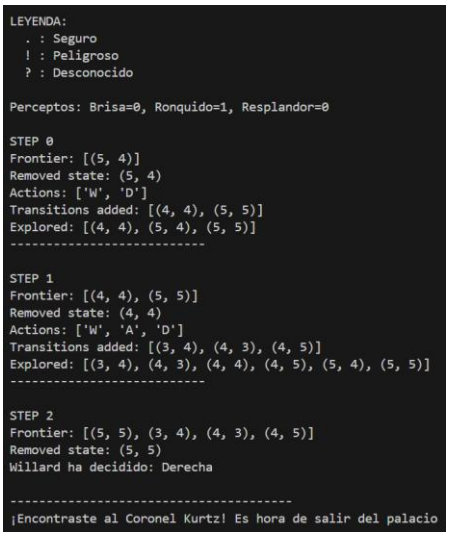
LEYENDA:
  . : Seguro
  ! : Peligroso
  ? : Desconocido

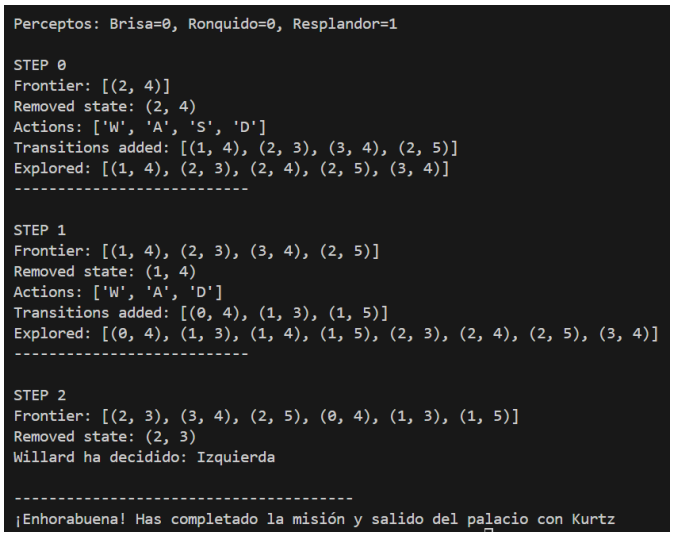
Perceptos: Brisa=1, Ronquido=0, Resplandor=0
Pulsa W/A/S/D para moverte, G+W/A/S/D para lanzar una granada o E para salir:
```

- **Modo automático.** En el que Willard decide sus acciones de forma autónoma. Para desplazarse, emplea **Búsqueda en Anchura (BFS)** restringida exclusivamente a celdas que conoce con certeza que son seguras.

Se ha elegido este algoritmo debido a que garantiza encontrar el camino más corto en un entorno con coste uniforme por movimiento, sin necesidad de heurísticas, y resulta especialmente adecuado en un entorno parcialmente observable donde la seguridad de las celdas es prioritaria.

El objetivo del agente es encontrar a Kurtz o alcanzar la salida (si ya le ha encontrado), para ello prioriza explorar celdas seguras no visitadas y, en caso de no existir, recurre al desplazamiento por celdas seguras ya visitadas para continuar la exploración. Cuando no existen acciones seguras que aporten nueva información, la ejecución finaliza para evitar decisiones arriesgadas que puedan conducir a la muerte del agente.





PARTE 2.1: Agente lógico-probabilístico para la búsqueda del Coronel Kurtz (palacio.py)

En esta segunda parte se aborda un problema de **toma de decisiones bajo incertidumbre** en un entorno **parcialmente observable**. El objetivo de Willard es localizar al Coronel Kurtz y, una vez encontrado, alcanzar la salida del palacio minimizando el riesgo de caer en trampas o ser capturado por un soldado enemigo.

Para ello, el agente combina **razonamiento probabilístico** mediante inferencia bayesiana con **búsqueda informada (A*)**, utilizando la información probabilística disponible para planificar trayectorias más seguras.

Modelado del entorno y perceptos

Al igual que en la primera parte, el palacio se modela como una cuadrícula de tamaño fijo, donde cada celda puede contener uno o **varios** elementos en este caso: Willard, el Coronel Kurtz, un soldado enemigo, trampas (fuego, pinchos y dardos) o la salida. El entorno es **estático**, ya que los elementos no cambian de posición, pero **parcialmente observable**, puesto que Willard no conoce directamente la ubicación real de los peligros ni de los objetivos y debe inferirla a partir de perceptos locales. Estos se representan como un vector booleano y proporcionan la misma información que los de la primera parte (trampas, soldado enemigo, salida, paredes del mapa y grito)

Estados, acciones y transiciones

El **estado del agente** viene determinado por: la posición actual de Willard, las celdas visitadas, las distribuciones de probabilidad asociadas a cada peligro (F, P, D, M, S) y variables internas como si Kurtz ha sido encontrado o si el agente sigue con vida. Se podría decir que el estado no es completamente observable, ya que el agente no conoce el estado real del entorno, sino una **creencia probabilística** sobre él.

Asimismo, las **acciones** disponibles para Willard, restringidas por los límites del palacio, son:

1. Moverse en una de las cuatro direcciones cardinales (arriba, abajo, izquierda y derecha)
2. Lanzar una granada a una celda adyacente (una única vez)
3. Abandonar la misión (solo en modo manual)

```
Pulsa W/A/S/D para moverte, G+W/A/S/D para lanzar una granada o E para salir:
```

En cuanto a las **transiciones**, son deterministas:

- Si Willard se mueve a una celda válida, su posición se actualiza.
- Si la celda contiene una trampa o un soldado, el juego termina.
- Si contiene a Kurtz, el objetivo intermedio se cumple.
- Si contiene la salida y Kurtz ha sido encontrado, la misión finaliza con éxito.

Inferencia bayesiana y actualización de creencias

Inicialmente, Willard asume **distribuciones uniformes** para todos los peligros y objetivos, excepto para la celda inicial, que se considera segura.

$$p(\tau_{ij}) = \frac{1}{N-1},$$

Cada vez que el agente recibe perceptos, actualiza sus creencias mediante una versión simplificada de **inferencia bayesiana**. Para cada celda se evalúa si se encuentra dentro del rango de influencia del percepto (distancia Manhattan <= 1) y se aplica una función de verosimilitud determinista:

- Si un peligro está en el rango y se percibe, la probabilidad se mantiene.
- Si no está en rango y no se percibe, también se mantiene (podría ser normal)
- En caso contrario, la probabilidad se descarta.
- Posteriormente, se normaliza la distribución para que la suma total sea 1

$$p(e_{ij}^{\tau} | \tau_{k\ell}) = \begin{cases} 1 & (k,l) \in adj(i,j) \cup \{(i,j)\}, \\ 0 & \text{otherwise.} \end{cases}$$

Mapa de riesgo

A partir de las distribuciones de probabilidad, el agente construye un **mapa de riesgo**, donde cada celda contiene la suma de las probabilidades de todos los peligros posibles. Este mapa representa el **coste esperado** de atravesar cada celda y es la base de la planificación informada.

```
 Mapa de PROBABILIDAD de TRAMPAS
 CW  0.00 0.09 0.09 0.09 0.09
0.00 0.09 0.09 0.09 0.09 0.09
0.09 0.09 0.09 0.09 0.09 0.09
0.09 0.09 0.09 0.09 0.09 0.09
0.09 0.09 0.09 0.09 0.09 0.09
0.09 0.09 0.09 0.09 0.09 0.09

 Mapa de PROBABILIDAD del SOLDADO
 CW  0.00 0.03 0.03 0.03 0.03
0.00 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03

 Mapa de PROBABILIDAD de la SALIDA
 CW  0.00 0.03 0.03 0.03 0.03
0.00 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03
```

```
 Mapa de PROBABILIDAD de TRAMPAS
0.00 0.00 0.00 0.10 0.10 0.10
0.00 1.00 0.00 0.10 0.10 0.10
0.00 0.00 0.00 0.00 0.10 0.10
0.00 0.00  CW  0.00 0.10 0.10
0.10 0.10 0.00 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10

 Mapa de PROBABILIDAD del SOLDADO
0.00 0.00 0.00 0.05 0.05 0.05
0.00 0.00 0.00 0.05 0.05 0.05
0.00 0.00 0.00 0.00 0.05 0.05
0.00 0.00  CW  0.00 0.05 0.05
0.05 0.05 0.00 0.05 0.05 0.05
0.05 0.05 0.05 0.05 0.05 0.05

 Mapa de PROBABILIDAD de la SALIDA
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.50 0.00 0.00 0.00
0.00 0.00 0.00 0.50 0.00 0.00
0.00 0.00  CW  0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
```

```
 Mapa de PROBABILIDAD de TRAMPAS
0.00 0.00 0.00 0.00 0.00 0.50
0.00 1.00 0.00 0.00 0.50  CW
0.00 0.00 0.00 0.00 1.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00

 Mapa de PROBABILIDAD del SOLDADO
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00  CW
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 1.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00

 Mapa de PROBABILIDAD de la SALIDA
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 1.00 0.00 0.00  CW
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
```

(Se observa como las probabilidades van actualizándose a medida que el agente de desplaza)

Búsqueda informada (A*)

Se plantea una búsqueda de caminos donde cada nodo representa una posición del agente y el coste acumulado g(n) es el riesgo total esperado del camino recorrido.

El objetivo es alcanzar primero al Coronel Kurtz y después la salida. Por ello, la planificación se realiza en dos fases: desde la posición inicial hasta Kurtz y, posteriormente, desde Kurtz hasta la salida.

Como heurística se utiliza la **distancia Manhattan** al objetivo. Esta heurística es admisible y consistente, ya que no sobreestima el coste real del camino y es independiente del riesgo (ya modelado en el coste acumulado). De este modo, el algoritmo A* prioriza caminos más seguros incluso si son más largos, reflejando una toma de decisiones conservadora bajo incertidumbre.

```
Planeando camino hasta Kurtz...

STEP 0
Frontier:
  Pos:(0, 0) g:0.00 h:3 f:3.00
Removed node: (0, 0)
Actions: ['Abajo', 'Derecha']
Transitions added:
  -> (1, 0) g:0.0 h:2 f:2.0
  -> (0, 1) g:0.0 h:4 f:4.0
Explored: [(0, 0)]
------------------------------

STEP 1
Frontier:
  Pos:(1, 0) g:0.00 h:2 f:2.00
  Pos:(0, 1) g:0.00 h:4 f:4.00
Removed node: (1, 0)
Actions: ['Abajo', 'Derecha']
Transitions added:
  -> (2, 0) g:0.12 h:1 f:1.12
  -> (1, 1) g:0.12 h:3 f:3.12
Explored: [(1, 0), (0, 0)]
------------------------------
```

```
STEP 0
Frontier:
  Pos:(3, 0) g:0.00 h:2 f:2.00
Removed node: (3, 0)
Actions: ['Arriba', 'Abajo', 'Derecha']
Transitions added:
  -> (2, 0) g:0.12 h:3 f:3.12
  -> (4, 0) g:0.12 h:1 f:1.12
  -> (3, 1) g:0.12 h:3 f:3.12
Explored: [(3, 0)]
------------------------------

STEP 1
Frontier:
  Pos:(2, 0) g:0.12 h:3 f:3.12
  Pos:(4, 0) g:0.12 h:1 f:1.12
  Pos:(3, 1) g:0.12 h:3 f:3.12
Removed node: (4, 0)
Actions: ['Abajo', 'Derecha']
Transitions added:
  -> (5, 0) g:0.24 h:0 f:0.24
  -> (4, 1) g:0.24 h:2 f:2.24
Explored: [(4, 0), (3, 0)]
------------------------------

STEP 2
Frontier:
  Pos:(2, 0) g:0.12 h:3 f:3.12
  Pos:(3, 1) g:0.12 h:3 f:3.12
  Pos:(5, 0) g:0.24 h:0 f:0.24
  Pos:(4, 1) g:0.24 h:2 f:2.24
Removed node: (5, 0)
Camino hasta la salida encontrado: [(3, 0), (4, 0), (5, 0)]
```

Ejecución

El sistema permite dos modos de funcionamiento:

- **Modo manual.** En el que el usuario decide las acciones apoyándose en mapas de probabilidad (heatmaps) de la existencia de trampas, del soldado y de la salida. Esto permite analizar visualmente el razonamiento probabilístico del agente y comprobar cómo las probabilidades se concentran a medida que se explora el entorno.
- **Modo automático**. En el que el agente planifica de forma autónoma toda la misión antes de ejecutarla, evitando zonas de alto riesgo

PARTE 2.2: River MDP con Value Iteration (river_mdp.py)

En esta parte del proyecto se modela un problema de toma de decisiones secuencial bajo incertidumbre mediante un **Proceso de Decisión de Markov (MDP)**. El entorno representa un río dividido en celdas donde el Capitán Willard y el Coronel Kurtz deben cruzar desde la orilla inicial hasta la orilla opuesta maximizando la recompensa total esperada, evitando peligros y teniendo en cuenta la presencia de corrientes.

El objetivo del agente es alcanzar la **casilla de salida**, situada en la última columna del mapa. La fila de esta se selecciona aleatoriamente en cada ejecución para introducir variabilidad en el entorno sin comprometer la existencia de trayectorias viables, obligando al agente a adaptarse a diferentes configuraciones.

Modelado del entorno como MDP

Cada **estado** viene determinado por la posición del agente en el mapa representada como (i, j) donde i es la fila y j la columna. El entorno está compuesto por los siguientes tipos de casillas:

- **R**: río seguro
- **I**: isla (obstáculo)
- **E**: salida
- **PEZ**: recompensa positiva
- **PIR**: piraña con penalización
- **" "**: orilla sin corriente

Desde cada estado el agente puede elegir entre las **acciones**: up, down, left y right. Si una acción conduce fuera del mapa o hacia una isla, el movimiento se considera inválido y el agente permanece en su posición actual, recibiendo igualmente el coste asociado.

Corriente y estocasticidad

Uno de los elementos clave del entorno es la **corriente del río**, modelada como una probabilidad dependiente de la columna. Para cada columna se define una probabilidad de ser arrastrado hacia abajo cuando el agente intenta moverse en cualquier otra dirección. Esto introduce **estocasticidad**, es decir, el agente no puede asumir que sus acciones tendrán siempre el efecto deseado, lo que le obliga a razonar en términos de **recompensa esperada** en lugar de trayectorias deterministas. En las columnas correspondientes a las orillas no existe corriente (zonas seguras sin arrastre).

Recompensas

La función de recompensa es fundamental para guiar el comportamiento del agente

- Cada movimiento tiene un **coste base de -1**, penalizando trayectorias excesivamente largas (buscamos soluciones eficientes).
- Las casillas **PEZ** proporcionan una recompensa positiva de **+5**, incentivando al agente a desviarse de rutas directas si el beneficio esperado así lo justifica
- Las casillas **PIR** generan una penalización de **-10**, representando un riesgo significativo que el agente debe intentar evitar
- La **salida** proporciona una recompensa muy alta de **+100**, asegurando que el objetivo final domine sobre recompensas intermedias.

Tanto los peces como las pirañas desaparecen tras ser visitados, evitando recompensas o castigos repetidos.

Por otro lado, $\gamma = 0.9$ se ha elegido para dar un peso elevado a las recompensas futuras sin ignorar las inmediatas, adecuado en un entorno donde las decisiones actuales pueden tener consecuencias importantes más adelante.

Algoritmo de Value Iteration y extracción de la política óptima

Para resolver el MDP se ha implementado **Value Iteration**, que permite calcular los valores óptimos de los estados resolviendo iterativamente las ecuaciones de Bellman.

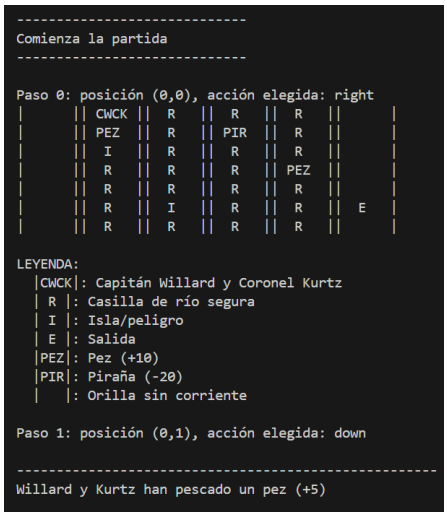
En cada iteración, el valor de un estado se actualiza como el máximo, sobre todas las acciones posibles, de la recompensa esperada más el valor descontado del estado siguiente. Además, tiene en cuenta la estocasticidad debida a la corriente, pues combina las probabilidades de moverse en la dirección deseada y de ser arrastrado.

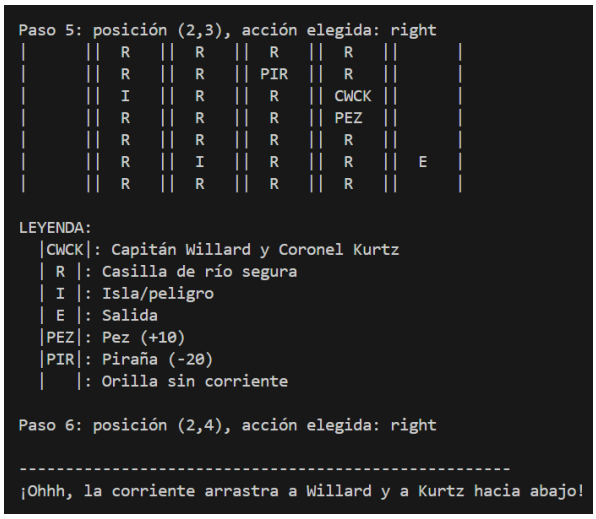
El proceso continúa hasta que el cambio máximo entre iteraciones es inferior a un umbral muy bajo $\theta = 0.001$, lo que indica que los valores han convergido.

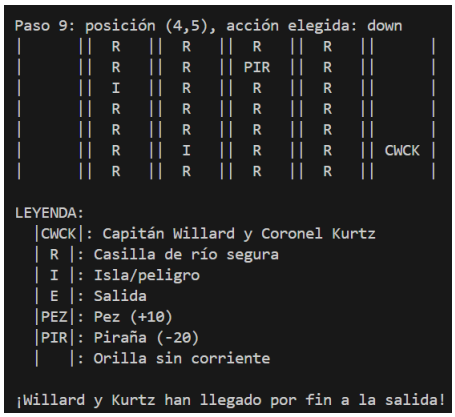
Una vez calculados los valores óptimos V(s) se extrae la **política óptima** seleccionando, para cada estado, la acción que maximiza la expectativa de recompensa utilizando dichos valores. La política resultante especifica la acción que el agente debería tomar para maximizar la recompensa esperada a largo plazo, teniendo en cuenta tanto los peligros como las recompensas del entorno.

Simulaciones

Mediante simulaciones se puede observar cómo el agente toma decisiones coherentes bajo incertidumbre (evita islas y pirañas) y, en muchos casos, opta por trayectorias ligeramente más largas cuando resulta beneficioso recoger peces.







Los resultados muestran claramente la influencia de las probabilidades de la corriente, la distribución de peligros y la presencia de recompensas adicionales.

Además, se introduce un límite máximo de pasos para evitar bucles infinitos.

Conclusión

El proyecto demuestra cómo un agente puede razonar y actuar eficazmente en entornos parcialmente observables y estocásticos combinando distintas técnicas. Desde el razonamiento lógico presente en la primera parte, pasando por la inferencia bayesiana y búsqueda informada, hasta la optimización de políticas mediante Value Iteration, el agente aprende a equilibrar riesgo y recompensa, mostrando un comportamiento coherente y prudente.